

## Higher Order Functions

### Q1: Match Maker

Implement `match_k`, which takes in an integer `k` and returns a function that takes in a variable `x` and returns `True` if all the digits in `x` that are `k` apart are the same.

For example, `match_k(2)` returns a one argument function that takes in `x` and checks if digits that are 2 away in `x` are the same.

`match_k(2)(1010)` has the value of `x = 1010` and digits 1, 0, 1, 0 going from left to right. `1 == 1` and `0 == 0`, so the `match_k(2)(1010)` results in `True`.

`match_k(2)(2010)` has the value of `x = 2010` and digits 2, 0, 1, 0 going from left to right. `2 != 1` and `0 == 0`, so the `match_k(2)(2010)` results in `False`.

**Important:** You may not use strings or indexing for this problem.

**Tip:** Floor dividing by powers of 10 gets rid of the rightmost digits.

```
def match_k(k):
    """Returns a function that checks if digits k apart match.

    >>> match_k(2)(1010)
    True
    >>> match_k(2)(2010)
    False
    >>> match_k(1)(1010)
    False
    >>> match_k(1)(1)
    True
    >>> match_k(1)(2111111111111111)
    False
    >>> match_k(3)(123123)
    True
    >>> match_k(2)(123123)
    False
    """
    def check(x):
        while x // (10 ** k) > 0:
            if (x % 10) != (x // (10 ** k)) % 10:
                return False
            x //= 10
        return True
    return check
```

# Trees

## Q2: In-order Traversal

Write a function that returns a generator that generates an “in-order” traversal, in which we yield the value of every node in order from left to right, assuming that each node has either 0 or 2 branches.

```
def in_order_traversal(t):
    """
    Generator function that generates an "in-order" traversal, in which we
    yield the value of every node in order from left to right, assuming that each node
    has either 0 or 2 branches.

    For example, take the following tree t:
        1
       2 3
      4 5
     6 7

    We have the in-order-traversal 4, 2, 6, 5, 7, 1, 3

    >>> t = Tree(1, [Tree(2, [Tree(4), Tree(5, [Tree(6), Tree(7)])]), Tree(3)])
    >>> list(in_order_traversal(t))
    [4, 2, 6, 5, 7, 1, 3]
    """
    if t.is_leaf():
        yield t.label
    else:
        left, right = t.branches
        yield from in_order_traversal(left)
        yield t.label
        yield from in_order_traversal(right)
```

# Iterators

## Q3: Repeated

Implement `repeated`, which takes in an iterator `t` and an integer `k` greater than 1. It returns the first value in `t` that appears `k` times in a row.

**Important:** Call `next` on `t` only the minimum number of times required. Assume that there is an element of `t` repeated at least `k` times in a row.

**Hint:** If you are receiving a `StopIteration` exception, your `repeated` function is calling `next` too many times.

```
def repeated(t, k):
    """Return the first value in iterator t that appears k times in a row,
    calling next on t as few times as possible.

    >>> s = iter([10, 9, 10, 9, 9, 10, 8, 8, 8, 7])
    >>> repeated(s, 2)
    9
    >>> t = iter([10, 9, 10, 9, 9, 10, 8, 8, 8, 7])
    >>> repeated(t, 3)
    8
    >>> u = iter([3, 2, 2, 2, 1, 2, 1, 4, 4, 5, 5, 5])
    >>> repeated(u, 3)
    2
    >>> repeated(u, 3)
    5
    >>> v = iter([4, 1, 6, 6, 7, 7, 8, 8, 2, 2, 2, 5])
    >>> repeated(v, 3)
    2
    """
    assert k > 1
    count = 0
    last_item = None
    while True:
        item = next(t)
        if item == last_item:
            count += 1
        else:
            last_item = item
            count = 1
        if count == k:
            return item
```

# Object-Oriented Programming

## Q4: Person

Modify the following `Person` class to add a `repeat` method, which repeats the last thing said. See the doctests for an example of its use.

```
class Person:
    """Person class.

    >>> steven = Person("Steven")
    >>> steven.repeat()          # initialized person has the below starting repeat phrase!
    'I squirreled it away before it could catch on fire.'
    >>> steven.say("Hello")
    'Hello'
    >>> steven.repeat()
    'Hello'
    >>> steven.greet()
    'Hello, my name is Steven'
    >>> steven.repeat()
    'Hello, my name is Steven'
    >>> steven.ask("preserve abstraction barriers")
    'Would you please preserve abstraction barriers'
    >>> steven.repeat()
    'Would you please preserve abstraction barriers'
    """

    def __init__(self, name):
        self.name = name
        self.previous = "I squirreled it away before it could catch on fire."

    def say(self, stuff):
        self.previous = stuff

        return stuff

    def repeat(self):
        return self.say(self.previous)

    def ask(self, stuff):
        return self.say("Would you please " + stuff)

    def greet(self):
        return self.say("Hello, my name is " + self.name)
```

# Linked Lists

## Q5: Every Other

Implement `every_other`, which takes a linked list `s`. It mutates `s` such that all of the odd-indexed elements (using 0-based indexing) are removed from the list. For example:

```
>>> s = Link('a', Link('b', Link('c', Link('d'))))
>>> every_other(s)
>>> s.first
'a'
>>> s.rest.first
'c'
>>> s.rest.rest is Link.empty
True
```

If `s` contains fewer than two elements, `s` remains unchanged. **Do not return anything!** `every_other` should mutate the original list.

```
def every_other(s):
    """Mutates a linked list so that all the odd-indexed elements are removed
    (using 0-based indexing).

    >>> s = Link(1, Link(2, Link(3, Link(4))))
    >>> every_other(s)
    >>> s
    Link(1, Link(3))
    >>> odd_length = Link(5, Link(3, Link(1)))
    >>> every_other(odd_length)
    >>> odd_length
    Link(5, Link(1))
    >>> singleton = Link(4)
    >>> every_other(singleton)
    >>> singleton
    Link(4)
    """
    if s is Link.empty or s.rest is Link.empty:
        return
    else:
        s.rest = s.rest.rest
        every_other(s.rest)
```

# Scheme

## Q6: Switch

Define the macro `switch`, which takes in an expression `expr` and a list of pairs called `cases` where the first element of the pair is some number and the second element is a single expression. `switch` will evaluate the expression contained in `cases` that corresponds to the number that `expr` evaluates to.

```
scm> (switch (+ 1 1) ((1 (print 'a))
                      (2 (print 'b))
                      (3 (print 'c))))
b
```

You may assume that the value `expr` evaluates to is always a number and is always the first element of one of the pairs in `cases`. You can also assume that the first value of each pair in `cases` is a number and the second expression does not contain the symbol `val`.

Use `equal?` to check if two numbers are equal.

For the example shown above, build the following expression:

```
(begin
  (define val (+ 1 1))
  (cond ((equal? val 1) (print 'a))
        ((equal? val 2) (print 'b))
        ((equal? val 3) (print 'c))))
```

This expression first assigns `val` to 3 and then compares `val` to the first element in each pair in `cases`.

```
(define-macro (switch expr cases)
  `(begin
    (define val ,expr)
    ,(cons
      'cond
      (map (lambda (case) (cons
        `(equal? val ,(car case))
        (cdr case)))
        cases))))
```

# SQL

## Q7: Raises

This last question refers to the following **salaries** table:

**salaries**

| name            | salary2022 | salary2023 |
|-----------------|------------|------------|
| Ben Bitdiddle   | 60000      | 80000      |
| Alyssa P Hacker | 40000      | 80000      |
| Cy D Fect       | 35000      | 74000      |
| Lem E Tweakit   | 25000      | 28000      |
| Louis Reasoner  | 30000      | 30000      |
| Oliver Warbucks | 150000     | 120000     |
| Eben Scrooge    | 75000      | 76000      |
| Robert Cratchet | 18000      | 20000      |
| Lana Lambda     | 610000     | 610000     |

Write a query that outputs the names of the top 3 employees with the largest salary raises from 2022 to 2023 along with their corresponding salary raises, ordered from largest to smallest raise.

```
SELECT name, salary2023 - salary2022
FROM salaries
ORDER BY salary2023 - salary2022 DESC
LIMIT 3
```



**celebrate your last discussion section!**